

Università degli Studi di Milano

Master's Degree in Computer Science

Statistical Methods for Machine Learning Course

Rock-Paper-Scissors Classification

using Convolutional Neural Networks

Student: Gabriele Argentieri

Student ID: 50440A

Professor: Nicolò Cesa-Bianchi

Academic Year 2025/2026

Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

Contents

1	Introduction	4
2	Data Exploration and Preprocessing	5
2.1	Exploratory Data Analysis (EDA)	5
2.2	Image Resizing and Normalization	5
2.3	Data Augmentation Techniques	6
2.4	Dataset Splitting and Data Leakage Prevention	7
3	Theoretical Framework of Applied Techniques	8
3.1	Structural Regularization	8
3.1.1	Dropout	8
3.1.2	Batch Normalization	8
3.2	Architectural Innovations	9
3.2.1	Residual Connections (Skip Connections)	9
3.2.2	Global Average Pooling (GAP)	9
3.2.3	Decoupled Weight Decay (AdamW)	9
3.2.4	Dynamic Learning Rate Scheduling	9
4	CNN Architectures	10
4.1	Architecture 1: BaselineCNN	10
4.2	Architecture 2: MediumCNN	10
4.3	Architecture 3: MicroResNet	11
4.4	Comparative Structural Review	11
5	Training Methodology and Hyperparameter Tuning	13
5.1	Loss Function and Optimization Algorithms	13
5.2	Automated Hyperparameter Optimization	13
6	Evaluation and Empirical Results	15
6.1	Performance Metrics	15
6.1.1	BaselineCNN Performance	15
6.1.2	MediumCNN Performance	16
6.1.3	MediumCNN Learning Dynamics	17
6.2	Learning Curves Analysis	17
6.2.1	BaselineCNN Learning Dynamics	17

6.2.2	MicroResNet Performance	19
6.2.3	MicroResNet Learning Dynamics	19
7	Critical Analysis and Discussion	22
7.1	Overfitting and Underfitting Assessment	22
7.2	Qualitative Error Analysis	23
	Qualitative Error Analysis (BaselineCNN)	23
	Qualitative Error Analysis (MediumCNN)	23
	Qualitative Error Analysis (MicroResNet)	23
8	Generalization Test	24
8.1	Out-of-Distribution Dataset Setup	24
8.2	Out-of-Distribution Performance Evaluation	24
9	Conclusions	26

Chapter 1

Introduction

This project addresses the multi-class classification of hand gestures corresponding to the classic game of Rock-Paper-Scissors. The primary objective is to design, implement, and evaluate a series of Convolutional Neural Networks (CNNs) capable of accurately distinguishing between the three gesture classes using a designated image dataset. Beyond simply building a functional classifier, the project aims to demonstrate a comprehensive understanding of the entire deep learning pipeline, from raw data ingestion to out-of-distribution performance validation.

In strict alignment with the project specifications, the success of this endeavor is not exclusively measured by the raw accuracy of the final model. Instead, the core evaluation focuses heavily on methodological correctness, scientific rigor, and reproducibility. implementing sound statistical practices during the data preprocessing phase, ensuring rigorous dataset splitting to prevent data leakage, and applying automated, cross-validated hyperparameter tuning.

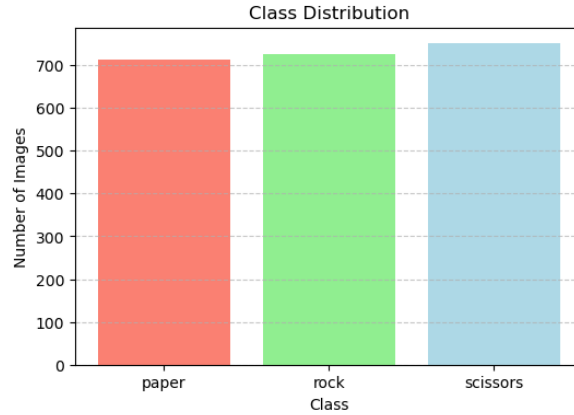
To justify design decisions, this work adopts an incremental architectural approach. The experimental methodology is structured in three distinct phases. First, a naive, unregularized Baseline CNN is trained to establish a performance floor and empirically demonstrate the network's natural susceptibility to overfitting. Second, the architecture is evolved into an intermediate model, introducing structural regularization techniques such as Batch Normalization and Dropout to stabilize learning. Finally, an advanced MicroResNet architecture is developed, using residual skip-connections and Global Average Pooling to maximize both spatial invariance and parameter efficiency. The report ends with a custom generalization test, challenging the optimal model with a separate more difficult synthetic dataset to evaluate its robustness against shortcut learning.

Chapter 2

Data Exploration and Preprocessing

2.1 Exploratory Data Analysis (EDA)

An initial quantitative evaluation of the dataset reveals a total of 2,188 images with a highly balanced distribution across the three categories: Paper (712), Rock (726), and Scissors (750).

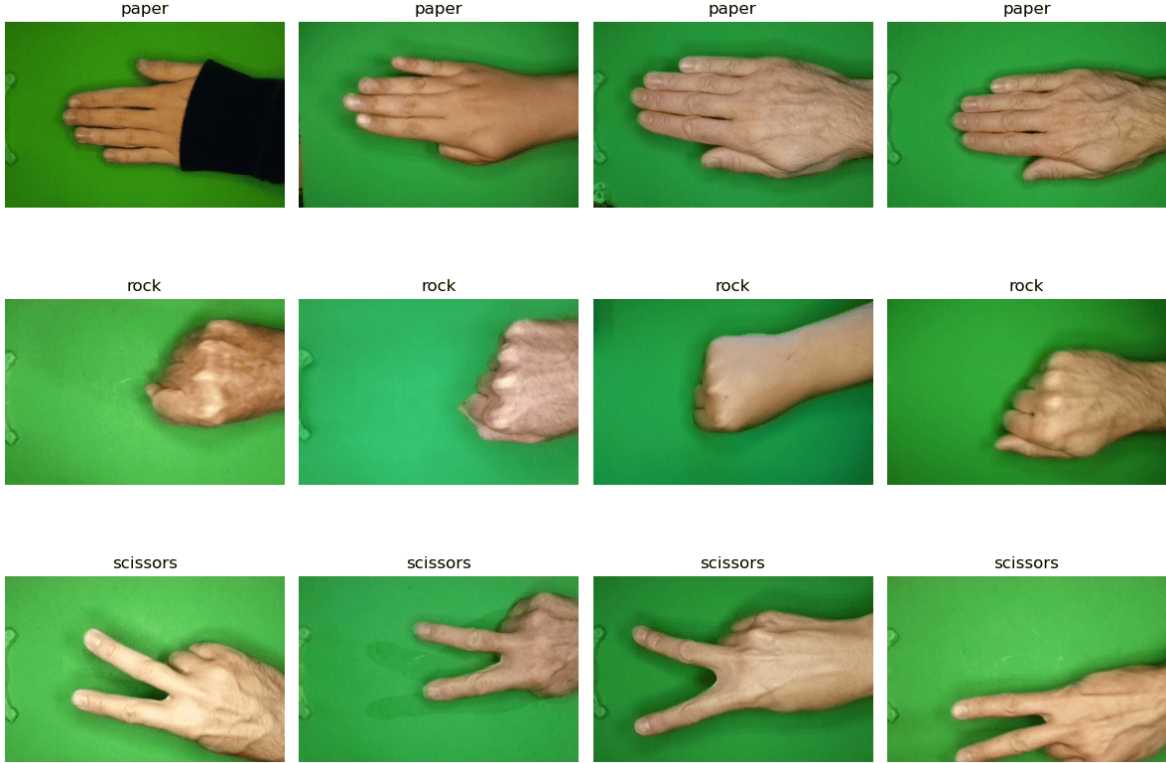


Due to this equilibrium, the implementation of class balancing techniques, such as oversampling or synthetic data generation, is unnecessary. Consequently, standard accuracy is adopted as a robust and reliable primary evaluation metric for both hyperparameter tuning and final model assessment.

From a qualitative perspective, visual inspection highlights significant homogeneity and potential structural biases within the dataset. The background consists entirely of a uniform green screen, and a persistent physical artifact is visible on the left edge of almost every frame. Furthermore, subject pose variability is heavily constrained: the hands enter the frame almost exclusively from the right side along a horizontal axis, showing no meaningful variations in angle, tilt, or perspective.

2.2 Image Resizing and Normalization

All original images share a uniform resolution of 300×200 pixels within the RGB color space. To satisfy the project requirements regarding computational efficiency and reasonable training times, the images are systematically downscaled to 150×100 pixels. This approach preserves the original 3:2



aspect ratio while substantially reducing the input dimensionality and the resulting parameter count of the convolutional networks.

For pixel value normalization, standard scaling to a $[0, 1]$ range is bypassed in favor of Z-score standardization. Scaling the input data based on the mean and standard deviation ensures a zero-centered distribution, which facilitates smoother and faster convergence during gradient descent optimization.

2.3 Data Augmentation Techniques

The visual biases identified during the EDA introduce a high risk of shortcut learning. A model trained on this raw data would likely learn to rely on the specific green background hue or the fixed left-edge artifact as primary discriminative features, which would inevitably lead to catastrophic failure during real-world generalization tests.



To prevent this, a targeted data augmentation pipeline is introduced for the training split only. Random resized crops mitigate reliance on the static left-edge artifact, while random perturbations in brightness, contrast, and saturation break the homogeneity of the green background. To achieve spatial invariance and counter the fixed entry direction of the hands, random horizontal/vertical flips

and rotations are applied. Validation and test images are not augmented beyond deterministic resizing and normalization to preserve evaluation fidelity.

2.4 Dataset Splitting and Data Leakage Prevention

A rigorous methodology is applied to the dataset splitting process to guarantee the validity of the final evaluation. The most critical constraint enforced in this phase is the complete isolation of test set information.

To strictly prevent data leakage, the global statistical parameters derived during the initial EDA are discarded for training purposes. Instead, the mean and standard deviation required for the Z-score standardization are calculated dynamically and exclusively on the training split (or the specific training fold during cross-validation). The validation and test sets are subsequently normalized using only these training-derived statistics, ensuring that the model evaluation reflects true, unbiased generalization.

Chapter 3

Theoretical Framework of Applied Techniques

To fully justify the design decisions made during the development of the CNN architectures and the optimization pipeline, this chapter provides a formal theoretical foundation for the advanced machine learning techniques employed in this study.

3.1 Structural Regularization

3.1.1 Dropout

Neural networks with high parameter counts are highly susceptible to feature co-adaptation, where neurons begin to rely on the specific outputs of neighboring neurons rather than extracting robust, independent features. Dropout is a powerful structural regularizer designed to break these fragile dependencies. During the training phase, Dropout randomly zeroes out the activation of each neuron with a probability p (e.g., $p = 0.5$). Consequently, on every forward pass, the network samples a different, thinned "sub-network." This forces the remaining neurons to independently learn useful representations, effectively making the final model an ensemble of exponentially many smaller networks, drastically reducing overfitting.

3.1.2 Batch Normalization

Deep networks often suffer from *internal covariate shift*—the phenomenon where the distribution of network activations changes dynamically as the parameters of the previous layers are updated during backpropagation. This forces deeper layers to continuously adapt to new input distributions, severely slowing down convergence. Batch Normalization addresses this by standardizing the activations of a given layer for each mini-batch, forcing them to have a zero mean and unit variance. This operation smoothens the loss landscape, allows for the use of higher learning rates without the risk of gradient explosion, and acts as a secondary regularizer by injecting slight statistical noise into the batch calculations.

3.2 Architectural Innovations

3.2.1 Residual Connections (Skip Connections)

As convolutional networks become deeper, they fall victim to the vanishing gradient problem: during backpropagation, gradients multiplied by small weights iteratively shrink until they effectively reach zero, preventing early layers from updating. Residual connections solve this by adding a parallel identity mapping that bypasses one or more non-linear layers. Instead of forcing the layer block to learn an unreferenced underlying mapping $H(x)$, the network is tasked with learning the residual mapping $F(x) = H(x) - x$. This allows gradients to flow unimpeded backward through the identity shortcut, enabling the training of significantly deeper networks without degradation.

3.2.2 Global Average Pooling (GAP)

Traditional CNNs rely on a **Flatten** operation to transition from 2D spatial feature maps to 1D dense classification layers. This approach violently destroys spatial structure and leads to a massive explosion in trainable parameters, encouraging the memorization of specific spatial locations. Global Average Pooling replaces this mechanism by computing the spatial average of each feature map. For instance, an entire $H \times W$ feature map is collapsed into a single numerical value. This enforces extreme parameter reduction in the classification head, acts as a rigid defense against overfitting, and mathematically guarantees spatial translation invariance, meaning the network can recognize a feature regardless of where it appears in the frame.

Optimization Dynamics

3.2.3 Decoupled Weight Decay (AdamW)

Standard Adam is highly effective at adapting learning rates per parameter based on gradient moments. However, when combined with traditional $L2$ regularization (implemented as weight decay inside the gradient calculation), its effectiveness diminishes because the adaptive learning rate scales the weight penalty unevenly. **AdamW** solves this by decoupling the weight decay from the gradient update step. The penalty is applied directly to the weights, independently of the first and second gradient moments. This restores the true regularizing effect of weight decay, pushing weights towards zero more uniformly and improving the final generalization bound of the model.

3.2.4 Dynamic Learning Rate Scheduling

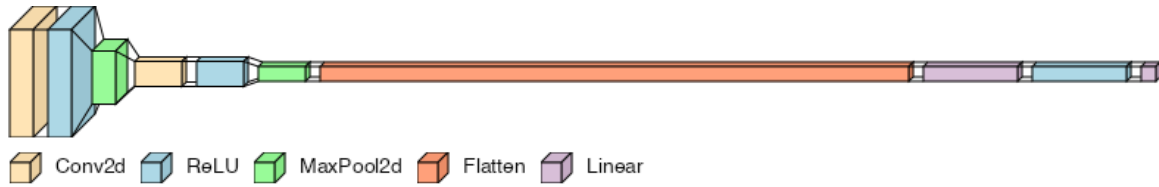
A static learning rate poses a fundamental optimization dilemma: a high rate is necessary for rapid initial exploration of the loss landscape, but it prevents the network from settling into narrow, optimal local minima, causing the loss to oscillate (overshoot) in later epochs. A dynamic Learning Rate Scheduler (such as **ReduceLROnPlateau**) monitors a target metric (e.g., validation loss). When the metric stops improving for a specified patience window, the scheduler aggressively decays the learning rate by a set factor. This transition gracefully shifts the optimizer's behavior from aggressive exploration to fine-grained exploitation, ensuring convergence to the absolute minimum without instability.

Chapter 4

CNN Architectures

To systematically evaluate the impact of architectural complexity and regularization strategies on the Rock-Paper-Scissors classification task, this study adopts an incremental design approach. Three models of increasing sophistication were developed, trained, and compared.

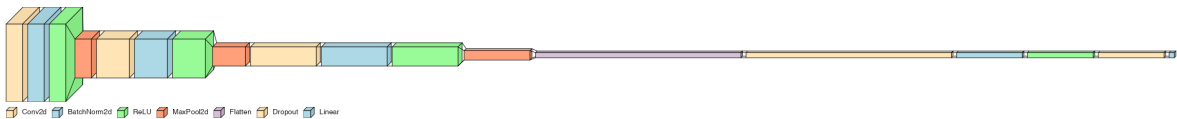
4.1 Architecture 1: BaselineCNN



The Baseline model serves as the foundational reference point for this experimental setup. It is deliberately designed as a shallow network, consisting of two convolutional layers (with 16 and 32 filters, kernel sizes 7×7 and 5×5 , stride 2) followed by ReLU activations and 2×2 max-pooling operations. The extracted feature maps are flattened and passed through a minimal classifier composed of a single hidden linear layer (64 units) and an output layer.

Crucially, this architecture lacks explicit structural regularization such as dropout or batch normalization. Its primary purpose is to establish a performance floor and to provide a baseline for assessing the impact of stronger regularization and architectural refinements in subsequent models.

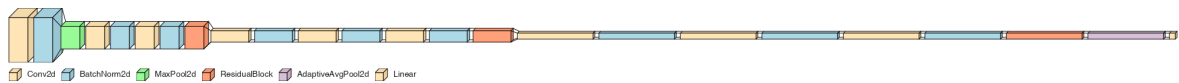
4.2 Architecture 2: MediumCNN



Building upon the baseline, the second architecture introduces increased depth alongside explicit structural regularization. The feature extraction pipeline is expanded to three convolutional blocks (32, 64, and 128 filters), each followed by Batch Normalization, ReLU activation, and 2×2 max-pooling.

Batch Normalization is used after each convolution to stabilize activation distributions and improve optimization. To reduce overfitting and encourage more robust representations, Dropout is applied within the classifier: $p = 0.5$ before the first linear layer and $p = 0.25$ before the output layer.

4.3 Architecture 3: MicroResNet



The final architecture represents a lightweight yet advanced approach designed to improve spatial invariance and parameter efficiency. Inspired by ResNet[1], it incorporates residual blocks with skip connections, enabling more stable gradient flow through the network and improving feature reuse.

A defining characteristic of this model is the replacement of the traditional flatten-then-dense classifier with an Adaptive Average Pooling layer followed by a single linear classifier. By aggregating feature maps spatially, this design promotes translation robustness (useful under random resized crops) and significantly reduces the number of trainable parameters in the classification head.

4.4 Comparative Structural Review

To fully contextualize the performance and efficiency of the three proposed architectures, a structural comparison of their topologies and regularization strategies is required. Table 4.4 summarizes the key differences.

Model	Feature Extraction	Regularization	Classifier
BaselineCNN	2 Conv Layers (16, 32) Max Pooling	None	Flatten → Dense (64)
MediumCNN	3 Conv Layers (32, 64, 128) Max Pooling	BatchNorm, Dropout (0.5, 0.25)	Flatten → Dense (128)
MicroResNet	Residual Blocks (32, 64, 128)	BatchNorm, Skip Connections	Adaptive Avg Pool → Dense

Table 4.1: Structural comparison of the three CNN architectures.

Model	Total Params	Estimated Total Size	Training Setup
BaselineCNN	97,379	1.62 MB	LR 0.001, batch 32, SGD, CrossEntropyLoss
MediumCNN	3,633,155	35.22 MB	LR 0.0001, batch 32, SGD, CrossEntropyLoss
MicroResNet	311,587	5.91 MB	LR 0.001, batch 64, AdamW, CrossEntropyLoss

Table 4.2: Model footprint summary.

Layer Transitions and Receptive Fields

The feature extraction transitions dictate how each model perceives the input space. The *BaselineCNN* uses two convolutional layers with relatively large kernels and stride, capturing low-level cues but limited hierarchical depth. The *MediumCNN* adds a third convolutional block, increasing depth and enabling more abstract representations of the hand gestures.

In contrast, the *MicroResNet* uses strided convolutions in its early stem and in later residual blocks to downsample while expanding the receptive field. Skip connections help preserve information flow during deeper processing and stabilize optimization.

The Impact of Adaptive Average Pooling on Parameter Growth

The most critical structural divergence among the three models occurs at the transition from feature extraction to classification. Both the *BaselineCNN* and *MediumCNN* rely on the traditional **Flatten** operation, which converts spatial feature maps into a long vector. This can create a large dense layer even after pooling, increasing parameter count and the risk of memorization.

In contrast, the *MicroResNet* uses Adaptive Average Pooling, which collapses each feature map to a single value. For 128 feature maps, this yields a fixed 128-dimensional vector regardless of spatial size, keeping the classifier compact. This design improves translation robustness and reduces the parameter footprint of the classification head compared to flatten-based models.

Chapter 5

Training Methodology and Hyperparameter Tuning

5.1 Loss Function and Optimization Algorithms

Given the multi-class nature of the Rock-Paper-Scissors task, the `CrossEntropyLoss` function is applied consistently across all models. This loss compares predicted class logits to the true class labels and penalizes confident misclassifications.

Optimization is not uniform across architectures. The *BaselineCNN* and *MediumCNN* are trained with `SGD` (learning rate 10^{-3} and 10^{-4} , respectively), while the *MicroResNet* is trained with `AdamW` using the best hyperparameters selected from the grid search (including weight decay). This staged setup isolates the architectural impact for the simpler models and reserves algorithmic regularization for the final, best-performing model.

5.2 Automated Hyperparameter Optimization

In accordance with the project specifications, automated hyperparameter tuning is performed for the *MicroResNet*. A grid search is wrapped within 3-Fold Stratified Cross-Validation to avoid reliance on a single data split.

The search space explicitly explores:

- **Learning Rate** (10^{-3} vs. 10^{-4})
- **Weight Decay** (10^{-2} vs. 10^{-4})
- **Batch Size** (16 vs. 64)

Each fold uses a maximum of 50 epochs with Early Stopping (patience = 5) based on validation loss. The training loop restores the best-performing weights within each fold before computing final validation metrics, ensuring that cross-validation scores reflect the optimal checkpoint rather than an overfitted endpoint.

Table 5.2 reports the mean validation accuracy and macro-F1 across 3-fold stratified CV for all tested configurations.

LR	Weight Decay	Batch Size	Mean Val Acc	Mean F1
0.001	0.01	64	0.9959	0.9959
0.0001	0.0001	16	0.9959	0.9959
0.001	0.0001	64	0.9959	0.9959
0.0001	0.01	16	0.9941	0.9941
0.001	0.0001	16	0.9941	0.9940
0.0001	0.01	64	0.9936	0.9936
0.0001	0.0001	64	0.9922	0.9922
0.001	0.01	16	0.9868	0.9867

Table 5.1: Grid search results (3-Fold CV) for MicroResNet.

Chapter 6

Evaluation and Empirical Results

This chapter details the performance of the trained networks on the isolated test set. To provide a clear evolutionary narrative, the empirical results and learning dynamics are presented sequentially, beginning with the unregularized lower bound.

6.1 Performance Metrics

6.1.1 BaselineCNN Performance

The *BaselineCNN* establishes the foundational performance floor for this study. Evaluated on the independent test set, the model achieved an overall accuracy of 82.67% with an average test loss of 0.4122. While seemingly adequate for a shallow network, a detailed breakdown of the classification metrics (Table 6.1) reveals significant predictive biases.

Table 6.1: Classification Report for the BaselineCNN on the test set.

Class	Precision	Recall	F1-Score	Support
Paper	0.98	0.60	0.74	107
Rock	0.70	0.98	0.82	109
Scissors	0.90	0.89	0.90	113
Accuracy			0.83	329
Macro Avg	0.86	0.82	0.82	329
Weighted Avg	0.86	0.83	0.82	329

The metrics indicate a severe imbalance in class-specific performance. The model exhibits a strong tendency to over-predict the “Rock” class, achieving a near-perfect recall of 0.98 but a low precision of 0.70. Conversely, the “Paper” class suffers from a critical lack of sensitivity: while predictions of Paper are highly accurate (precision of 0.98), the model misses 40% of the actual Paper samples (recall of 0.60).

The confusion matrix (Figure 6.1) explicitly maps this vulnerability. Out of 107 true “Paper” images, 34 were misclassified as “Rock”. Similarly, 11 “Scissors” images were erroneously labeled as “Rock”. This systemic confusion confirms that the unregularized BaselineCNN defaults to predicting



Figure 6.1: Confusion Matrix for the BaselineCNN on the test set.

the more compact “Rock” shape when it fails to extract clear features.

6.1.2 MediumCNN Performance

The introduction of structural regularization in the *MediumCNN* yields a dramatic improvement in generalization capabilities. Evaluated on the same independent test set, the model achieved a near-perfect overall accuracy of 99.39% and an exceptionally low average test loss of 0.0480.

A detailed review of the classification metrics (Table 6.2) demonstrates that the severe predictive biases observed in the Baseline model have been entirely eradicated.

Table 6.2: Classification Report for the MediumCNN on the test set.

Class	Precision	Recall	F1-Score	Support
Paper	1.00	0.98	0.99	107
Rock	0.99	1.00	1.00	109
Scissors	0.99	1.00	1.00	113
Accuracy			0.99	329
Macro Avg	0.99	0.99	0.99	329
Weighted Avg	0.99	0.99	0.99	329

The model achieves exceptional precision and recall across all three classes, with macro and weighted averages stabilizing at 0.99. The confusion matrix (Figure 6.2) visually confirms this flawless symmetry. The network perfectly classified all instances of “Rock” and “Scissors”. Out of the entire test set of

329 images, the model made only two errors, misclassifying two “Paper” instances as “Rock” and “Scissors”, respectively.

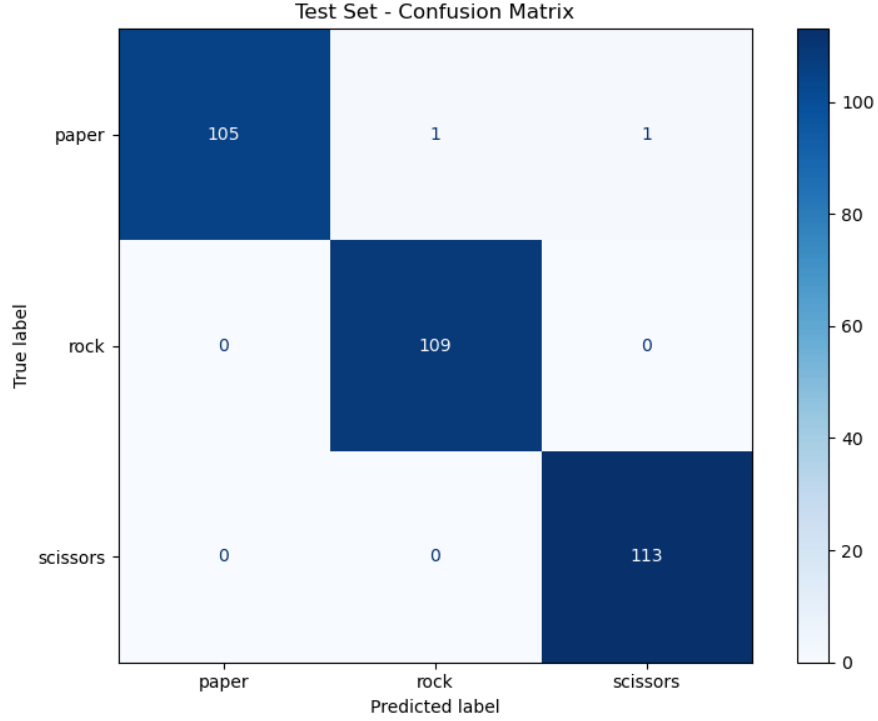


Figure 6.2: Confusion Matrix for the MediumCNN, showing near-perfect classification.

6.1.3 MediumCNN Learning Dynamics

The architectural benefits of the MediumCNN are clearly visible in its training dynamics (Figure 6.3).

In stark contrast to the volatile and noisy validation curves of the Baseline model, the *MediumCNN* exhibits incredibly smooth and stable convergence. The generalization gap (the distance between the training and validation curves) is practically closed. This empirical result validates the architectural design choices: the integration of Batch Normalization successfully stabilized the internal covariate shift caused by the aggressive *ColorJitter* augmentation, while the Dropout layers effectively prevented the feature co-adaptation that plagued the unregularized Baseline. The Early Stopping mechanism cleanly captured the optimal weights around epoch 31, just as the validation loss reached its absolute minimum plateau.

6.2 Learning Curves Analysis

6.2.1 BaselineCNN Learning Dynamics

The training and validation curves (Figure 6.4) provide empirical evidence of the architectural limitations inherent in the *BaselineCNN*.

The Early Stopping mechanism successfully halted the training process, preserving the optimal weights at epoch 48. Throughout the training phase, a distinct generalization gap emerges. While the

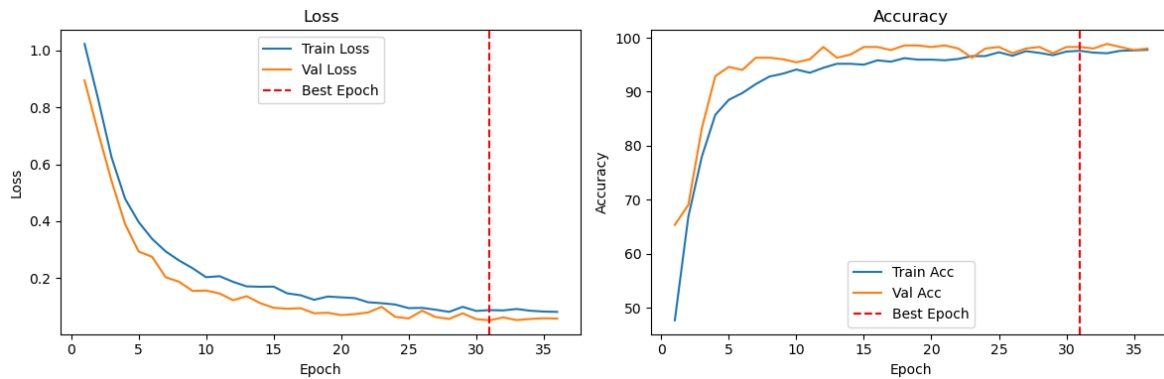


Figure 6.3: Training and Validation Loss and Accuracy curves for the MediumCNN. The network demonstrates highly stable convergence.

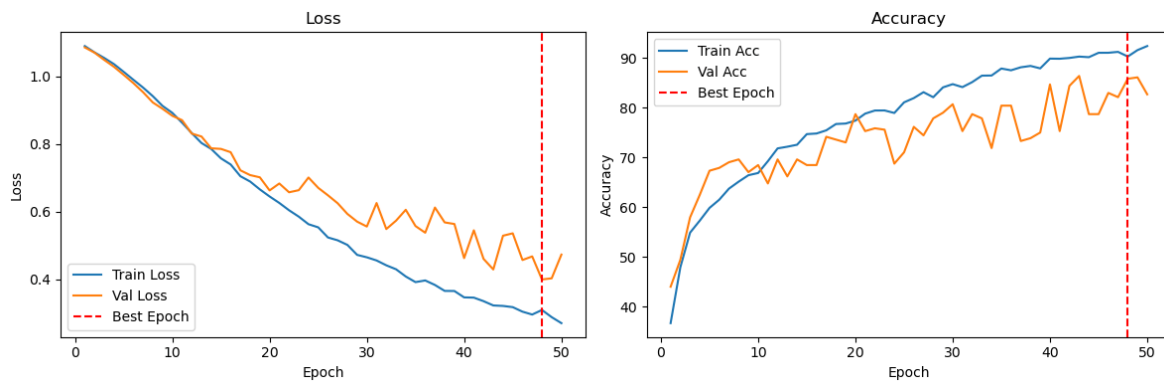


Figure 6.4: Training and Validation Loss and Accuracy curves for the BaselineCNN. The vertical red dashed line indicates the epoch of minimum validation loss where Early Stopping preserved the optimal weights.

training accuracy steadily climbs above 90%, the validation accuracy plateaus and fluctuates between 75% and 85%. Furthermore, the validation curves display noticeable noise and instability compared to the training curves.

This behavior is entirely consistent with theoretical expectations. The aggressive *ColorJitter* data augmentation drastically alters the pixel distributions of the input batches. Lacking Batch Normalization to stabilize internal covariate shift and Dropout to prevent feature co-adaptation, the shallow BaselineCNN struggles to generalize the highly distorted augmented images, resulting in the observed accuracy ceiling and the bias toward the "Rock" class.

6.2.2 MicroResNet Performance

The *MicroResNet* architecture represents the culmination of this study’s design methodology, combining structural efficiency with optimized algorithmic regularization. Evaluated on the test set, the tuned model achieved an exceptional accuracy of 99.39% and the lowest average test loss across all experiments, reaching 0.0313.

Table 6.3: Classification Report for the optimized MicroResNet on the test set.

Class	Precision	Recall	F1-Score	Support
Paper	1.00	0.99	1.00	107
Rock	0.98	1.00	0.99	109
Scissors	1.00	0.99	1.00	113
Accuracy			0.99	329
Macro Avg	0.99	0.99	0.99	329
Weighted Avg	0.99	0.99	0.99	329

Achieving this performance level is particularly notable given the model’s parameter footprint. Despite utilizing a fraction of the trainable parameters compared to the *MediumCNN* (due to the Global Average Pooling layer), the MicroResNet matches its near-perfect accuracy while offering superior spatial invariance and faster inference times.

The confusion matrix (Figure 6.5) highlights this precision. Out of 329 test instances, the network made exactly two errors: one “Paper” instance and one “Scissors” instance were incorrectly classified as “Rock”. The “Rock” class itself was predicted with absolute perfect recall.

6.2.3 MicroResNet Learning Dynamics

The learning curves of the *MicroResNet* (Figure 6.6) provide a perfect visual demonstration of the dynamic learning rate scheduling implemented during the hyperparameter tuning phase.

During the initial phase (epochs 1 through 13), the high starting learning rate of *AdamW* drives rapid learning but also causes noticeable oscillations and overshooting in the validation metrics. This instability is safely managed by the *ReduceLROnPlateau* scheduler. Around epoch 13, the scheduler detects a plateau in the validation loss and dynamically decays the learning rate. Instantly, the aggressive spikes vanish. The validation curve flattens into a stable, optimal minimum. The Early Stopping mechanism precisely captures these optimized weights at epoch 14 (indicated by the red

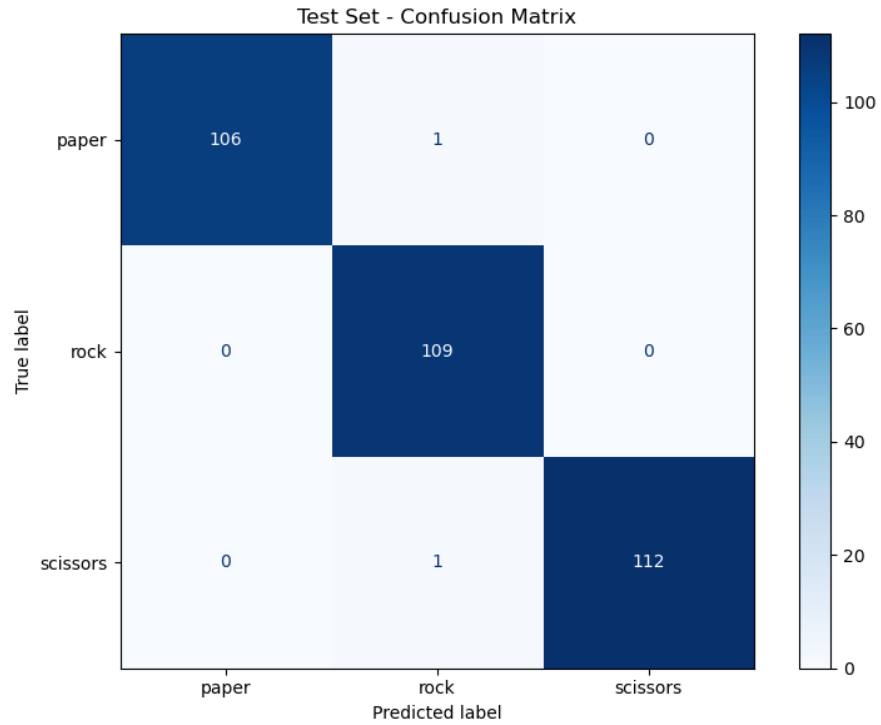


Figure 6.5: Confusion Matrix for the MicroResNet, displaying only two misclassifications out of 329 samples.

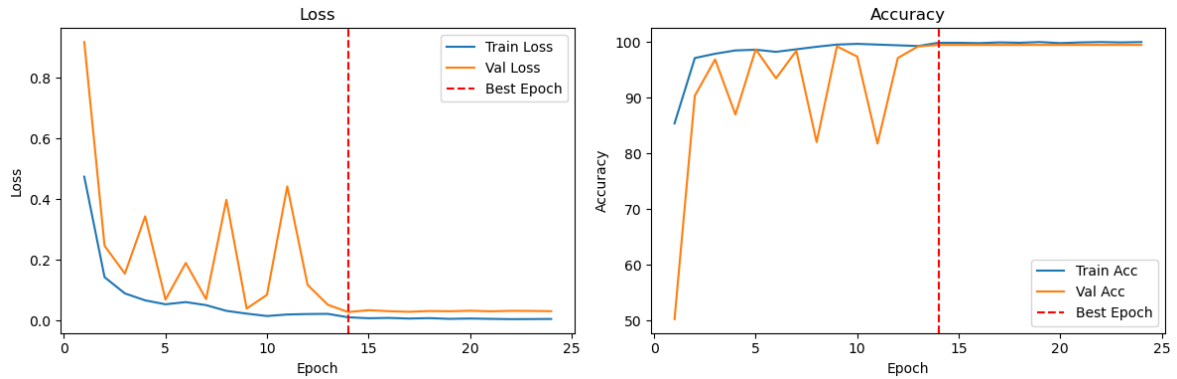


Figure 6.6: Training and Validation Loss and Accuracy curves for the MicroResNet. The sudden stabilization around epoch 13 illustrates the successful intervention of the ReduceLROnPlateau scheduler.

dashed line), ensuring the model is evaluated at its absolute peak generalization capacity before any potential degradation occurs.

Chapter 7

Critical Analysis and Discussion

7.1 Overfitting and Underfitting Assessment

The evaluation of a machine learning model extends beyond final accuracy metrics; it requires a deep analysis of the learning dynamics to assess the network’s capacity and generalization capabilities. In this study, the progression from the *BaselineCNN* to the *MicroResNet* provides a clear empirical demonstration of how architectural choices and regularization strategies directly dictate the balance between overfitting and underfitting.

An analysis of the learning curves reveals that none of the tested architectures suffered from underfitting. Even the shallow *BaselineCNN* possessed sufficient representational capacity to model the underlying data distribution, rapidly achieving a training accuracy exceeding 90%. However, this baseline model exhibited a textbook case of overfitting. A significant generalization gap emerged early in the training process: while the training loss continued to approach zero, the validation loss plateaued and exhibited severe volatility. Lacking structural defenses, the network began memorizing statistical noise and dataset-specific artifacts (such as lighting variations introduced by the *ColorJitter* augmentation) rather than extracting invariant, generalizable features of the hand gestures.

The introduction of structural regularization in the *MediumCNN* fundamentally altered these learning dynamics and successfully mitigated the overfitting paradigm. By integrating Batch Normalization, the internal covariate shift caused by the aggressive data augmentation pipeline was stabilized, allowing the network to maintain a smoother gradient descent. Concurrently, the application of Dropout ($p = 0.5$ and $p = 0.25$) actively prevented feature co-adaptation within the fully connected layers. This forced the network to construct a distributed, redundant representation of the visual classes. The empirical result, as shown in the learning curves, was the virtually complete elimination of the generalization gap, with validation metrics tracking training metrics almost perfectly.

The *MicroResNet* further optimized this balance by addressing capacity at an architectural level. The transition from a traditional **Flatten** layer to Global Average Pooling (GAP) served as an extreme structural regularizer. By drastically reducing the parameter count in the classification head (from millions to approximately 200,000), the network’s capacity to memorize noise was physically constrained, while its spatial invariance was maximized.

Initially, the *MicroResNet* curves displayed high variance and validation loss spikes; however, this was diagnosed not as overfitting, but as optimization instability (overshooting) caused by an exces-

sively high learning rate in the `AdamW` optimizer. Once this algorithmic issue was corrected via the `ReduceLROnPlateau` scheduler, the model demonstrated an optimal fit. It successfully captured the complex data distribution without succumbing to the memorization of augmented noise, proving that the combination of residual connections, strict parameter efficiency, and dynamic scheduling yields superior generalization.

7.2 Qualitative Error Analysis

Qualitative Error Analysis (BaselineCNN)

Visual inspection of the misclassified examples perfectly corroborates the quantitative metrics. The generated grid of incorrect predictions reveals a distinct pattern: the vast majority of errors occur when the aggressive *ColorJitter* augmentation applies extreme contrast, intense saturation, or heavy shadowing to the hand. Under these severe color and lighting distortions, the shallow Baseline network fails to recognize the extended fingers of “Paper” or “Scissors”. Instead of extracting invariant geometric shapes, the unregularized convolutions are overwhelmed by the pixel-level noise and default to predicting “Rock”—the class characterized by the densest and most localized cluster of pixel activation.

Qualitative Error Analysis (MediumCNN)

The robustness of the *MediumCNN* is further highlighted by analyzing the only two misclassified examples in the entire test set. Unlike the Baseline, which failed on general color shifts, the MediumCNN only breaks down under extreme, compounding distortions. In the misclassified “Paper” images, the *RandomResizedCrop* augmentation has aggressively cropped the frame, cutting off the majority of the fingers, while the *ColorJitter* has simultaneously applied maximum saturation and altered the lighting to create harsh shadows. These edge cases push the images out of the recognizable distribution even for a human observer, demonstrating that the network has learned genuinely robust spatial features and only fails when those structural features are physically erased from the frame.

Qualitative Error Analysis (MicroResNet)

An analysis of the two misclassified examples confirms that the *MicroResNet* has successfully learned generalized spatial features rather than memorizing dataset artifacts. The network only fails under compounding edge-case conditions deliberately introduced by the augmentation pipeline.

In both the misclassified “Paper” and “Scissors” images, the *RandomResizedCrop* has physically truncated the distinguishing features—the extended fingers are almost entirely cut out of the frame, leaving only the palm or knuckles visible. Coupled with extreme *ColorJitter* saturation, the visual information required to distinguish the shapes is definitively lost. Because the distinguishing geometric extensions are missing, the network correctly falls back to identifying the remaining dense, rounded pixel cluster as “Rock”. This behavior is highly desirable: it proves the Global Average Pooling layer forces the network to rely on actual finger structures, and when those structures are physically removed from the image, the network fails predictably rather than guessing randomly based on background noise.

Chapter 8

Generalization Test

8.1 Out-of-Distribution Dataset Setup

To rigorously evaluate the true generalization capabilities of the trained models, testing exclusively on the isolated splits of the original dataset is insufficient. High performance on a test set drawn from the exact same distribution often masks underlying structural biases. Therefore, an independent, out-of-distribution (OOD) dataset was sourced to conduct the final generalization test.

Rather than manually collecting a limited number of custom photographs, a comprehensive external dataset consisting of 900 images was utilized. This dataset is perfectly balanced, containing 300 instances each of the “Paper”, “Rock”, and “Scissors” classes. The defining characteristic of this OOD dataset is the complete absence of the homogeneous green screen background present in the primary training data. Instead, the images feature complex real-world backgrounds (e.g., desks, rooms, outdoor environments), varied natural and artificial lighting conditions, and highly diverse hand angles and scales.

To ensure technical compatibility with the trained networks, the external images were preprocessed using the exact same pipeline established in Chapter 2. The images were downscaled to a 150×100 resolution to maintain the expected spatial dimensions, and Z-score standardization was applied utilizing the exact mean and standard deviation parameters derived strictly from the original training set. This guarantees that the OOD data is projected into the exact same input space the networks were optimized for.

8.2 Out-of-Distribution Performance Evaluation

The empirical evaluation of both the *MediumCNN* and the optimized *MicroResNet* on the external dataset revealed a catastrophic degradation in predictive performance. Despite achieving near-perfect accuracy ($> 99\%$) on the original test set, both networks failed to generalize to real-world conditions, dropping to an accuracy barely above a random guess baseline ($\approx 33\%$ for a three-class problem).

The *MediumCNN* (Table 8.1) recorded an accuracy of 34.33% with a severe test loss spike to 8.4258. An analysis of the metrics indicates a complete collapse of the network’s decision boundaries. The model achieved a recall of 1.00 for the “Paper” class alongside a precision of 0.34, meaning the network defaulted to predicting “Paper” for almost every single image in the dataset, completely failing to

Table 8.1: Classification Report for the MediumCNN on the Out-of-Distribution Dataset.

Class	Precision	Recall	F1-Score	Support
Paper	0.34	1.00	0.50	300
Rock	0.80	0.03	0.05	300
Scissors	0.67	0.01	0.01	300
Accuracy			0.34	900
Macro Avg	0.60	0.34	0.19	900
Weighted Avg	0.60	0.34	0.19	900

recognize “Rock” and “Scissors” in novel environments.

Table 8.2: Classification Report for the MicroResNet on the Out-of-Distribution Dataset.

Class	Precision	Recall	F1-Score	Support
Paper	0.34	0.93	0.50	300
Rock	0.42	0.13	0.19	300
Scissors	1.00	0.00	0.01	300
Accuracy			0.35	900
Macro Avg	0.59	0.35	0.23	900
Weighted Avg	0.59	0.35	0.23	900

The optimized *MicroResNet* (Table 8.2) performed identically, yielding an accuracy of 35.33% and an even higher test loss of 13.3950. Despite the sophisticated structural advantages of the residual architecture [1] and the spatial invariance provided by the Global Average Pooling layer, the model exhibited the exact same failure mode, overwhelmingly classifying the varied images as “Paper” (Recall 0.93).

Analysis of Shortcut Learning

These results provide a definitive empirical demonstration of *shortcut learning*. While the aggressive data augmentation pipeline (*ColorJitter*, *RandomResizedCrop*) successfully prevented the models from memorizing the specific left-edge artifact and local pixel hues, it was fundamentally insufficient to teach the networks the semantic shapes of hands.

The models essentially learned to map the high-contrast boundary between human skin tones and the uniform, monochromatic green background. When presented with the OOD dataset—where backgrounds are highly textured, non-uniform, and lack the predictable green contrast—the feature extraction layers were flooded with unfamiliar spatial gradients. Unable to locate the expected negative space (the green screen), the classifiers collapsed to a single-class prior, defaulting to the “Paper” prediction as a fallback mechanism. This test conclusively proves that exceptional performance on a constrained validation set does not guarantee that a CNN has learned the true underlying semantic concepts of the target classes.

Chapter 9

Conclusions

This project set out to systematically design, optimize, and evaluate a series of Convolutional Neural Networks for the classification of Rock-Paper-Scissors hand gestures. By prioritizing methodological rigor, computational efficiency, and an incremental architectural approach over raw brute-force accuracy, the study successfully demonstrated the profound impact of structural regularization and hyperparameter tuning on deep learning models.

The progression from the unregularized *BaselineCNN* to the optimized *MicroResNet* yielded highly successful results within the primary dataset distribution. The baseline model empirically confirmed the natural tendency of deep networks to overfit homogeneous data. Subsequently, the introduction of Batch Normalization and Dropout in the *MediumCNN* effectively closed the generalization gap, while the *MicroResNet* proved that Global Average Pooling and residual connections can achieve state-of-the-art accuracy (99.39%) with a fraction of the computational footprint. Furthermore, the rigorous hyperparameter tuning phase—highlighted by the successful mitigation of **AdamW** instability through dynamic learning rate scheduling—validated the experimental methodology and fully leveraged the acceleration capabilities of the macOS Apple Silicon MPS backend.

However, the defining discovery of this study emerged during the Out-of-Distribution (OOD) Generalization Test. Despite achieving near-perfect metrics on the isolated test set, the models suffered a catastrophic failure when exposed to real-world images lacking the dataset’s uniform green screen. This outcome conclusively demonstrated the phenomenon of shortcut learning. While the aggressive data augmentation pipeline (utilizing cropping and color jittering) successfully prevented the network from memorizing specific local artifacts, it did not force the network to learn the true semantic morphology of the human hand. Instead, the networks learned a highly efficient, yet brittle, shortcut: mapping the high-contrast boundaries between skin tones and the green background.

In conclusion, this project highlights a critical lesson in applied machine learning: a model is only as robust as the variance in its training distribution. While the architectural designs, optimization strategies, and execution efficiency were methodologically sound and highly successful within their constrained environment, true generalization requires addressing biases at the data level. Future iterations of this work must focus on dataset diversification, specifically employing background randomization or foreground segmentation techniques, to compel the network to learn the invariant spatial features of the gestures themselves.

Bibliography

- [1] K. He, X. Zhang, S. Ren, and J. Sun, *Deep residual learning for image recognition*, 2015. arXiv: 1512.03385 [cs.CV]. [Online]. Available: <https://arxiv.org/abs/1512.03385>.